

Project Report: Enhancing LLM Inference with GraphRAG

Daniel Granström
Aalto University
Espoo, Finland
daniel.granstrom@aalto.fi

Joonatan Korpela
Aalto University
Espoo, Finland
joonatan.korpela@aalto.fi

Mikael Siidorow
Aalto University
Espoo, Finland
mikael.siidorow@aalto.fi

Abstract

The course project is available at <https://github.com/dcgran/CS-E4780-project> [1].

- GraphRAG implementation on Nobel Laureates dataset
- Text2Cypher accuracy improvements + performance optimization
- Key metrics: accuracy, latency, cache hit rate, memory usage
- GitHub repository: <https://github.com/dcgran/CS-E4780-project> [1]

Keywords

Large-Language Models

ACM Reference Format:

Daniel Granström, Joonatan Korpela, and Mikael Siidorow. 2025. Project Report: Enhancing LLM Inference with GraphRAG. In *Proceedings of CS-E4780 Course Project Evaluation Tables (CS-E4780)*. ACM, New York, NY, USA, 2 pages.

1 Introduction

- Graph-based RAG vs traditional RAG background
- Nobel Laureates dataset overview
- Task 1: Text2Cypher enhancement
- Task 2: Caching and performance benchmarking
- Report organization

2 System Architecture

- Overall GraphRAG pipeline diagram
- Knowledge graph schema (Kuzu)
- Component interactions
- Design decisions and rationale for each task

3 Implementation

3.1 Testbed Setup

- Hardware specifications
- Software environment (Python, Kuzu, LLM API)
- Test query dataset

3.2 Task 1: Text2Cypher Enhancement

- Dynamic few-shot exemplar selection
- Self-refinement loop (generate → validate → repair)
- Rule-based post-processing
- Implementation details for each component

3.3 Task 2: Caching and Performance

- LRU cache design (key computation, data structure)
- Benchmarking infrastructure
- Memory measurement methodology (process-level, object-level, database-level)

4 Performance Evaluation

4.1 Task 1: Text2Cypher Accuracy and Latency

Table 1 presents the ablation study showing accuracy-latency trade-offs.

- Analysis of accuracy-latency trade-offs
- Which combinations are most effective?
- Example success/failure cases

4.2 Task 2: Performance and Efficiency

Table 2 shows the pipeline stage latency breakdown.

Table 3 shows the memory utilization breakdown.

- Cache hit rate analysis
- Speedup quantification
- Performance visualization (flamegraph/bar chart)
- Bottleneck identification

5 Discussion

- Key findings from ablation study
- Optimal configuration recommendation
- Challenges encountered and solutions
- Lessons learned

6 Conclusion

- Summary of Task 1 and Task 2 achievements
- Key contributions
- Practical insights

References

- [1] Daniel Granström, Joonatan Korpela, and Mikael Siidorow. 2025. Enhancing LLM Inference with GraphRAG. <https://github.com/dcgran/CS-E4780-project>. Accessed: 2025-10-10.

Table 1: Ablation Study - Accuracy vs Latency Trade-offs

Configuration	Syntax Correct (%)	Semantic Correct (%)	Avg Latency (ms)	Avg Iterations
Baseline (static few-shot)	XX	XX	XXX	1.0
Only Few-shot selection	XX	XX	XXX	1.0
Only Self-refinement	XX	XX	XXX	X.X
Only Post-processing	XX	XX	XXX	1.0
Few-shot + Post-processing	XX	XX	XXX	1.0
Few-shot + Refinement	XX	XX	XXX	X.X
Refinement + Post-processing	XX	XX	XXX	X.X
All enhancements	XX	XX	XXX	X.X

Table 2: Pipeline Stage Latency Breakdown (ms)

Stage	Without Cache	With Cache (hit)	With Cache (miss)
Schema Pruning	XX	XX	XX
Text2Cypher	XXX	<1	XXX
Query Execution	XX	XX	XX
Response Generation	XX	XX	XX
Total	XXX	XX	XXX

Table 3: Memory Utilization Breakdown (MB)

Component	Memory	Measurement Method
Kuzu database	XX	psutil RSS
Python runtime	XX	psutil RSS
LRU cache	XX	pymppler.sizeof
Embedding model	XX	Model footprint
Peak working set	XX	Max RSS